

Term 2 Notes: Applications Development Practice 4 (ADP470S)

Advanced Diploma in ICT

Cape Peninsula University of Technology

2026

Overview of Term 2

Term 2 builds on Term 1 foundations (basic algorithms, complexity, recursion, searching, basic sorting, arrays & linked lists).

Key Focus Areas:

- Review & deeper recursion + Stack
- Queue variants (including Circular & Priority)
- Binary Trees & Traversals
- Height-Balanced Trees (AVL)
- **Sorting Algorithms + Rigorous Analysis** (major emphasis)

Assessment: T2 Written Test (35% of final mark) + ongoing exercises.

1. Review of Recursion & Stack Operations (SO3)

Recursion Recap (from Term 1):

- A function that calls itself.
- Must have **base case** + **recursive case**.
- Examples: Factorial, Fibonacci.

Stack Connection:

- Recursion uses the **call stack** implicitly.
- Every recursive call pushes a new frame (local variables, return address).
- When base case is reached, frames are popped (unwinding).

Explicit Stack Operations (LIFO - Last In First Out):

Python

```
# Basic Stack (can be implemented with list or Linked List)
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop() if not self.empty() else None

    def peek(self):
        return self.items[-1] if not self.empty() else None

    def empty(self):
        return len(self.items) == 0
```

Applications of Stack:

- Function call management (Recursion)
- Expression evaluation (Infix → Postfix)
- Backtracking (Maze, Sudoku)
- Undo/Redo functionality
- Browser history

Example: Balanced Parentheses using Stack

Python

```
def is_balanced(expr):
    stack = Stack()
    pairs = {'(': ')', '{': '}', '[': ']'}
    for char in expr:
        if char in '({[':
            stack.push(char)
        elif char in ')}]':
            if stack.empty() or stack.pop() != pairs[char]:
                return False
    return stack.empty()
```

2. Queue Operations & Circular Queue (SO4)

Queue: FIFO (First In First Out)

Basic Operations:

- enqueue(item)
- dequeue()
- front() / rear()
- isEmpty()
- size()

Implementation Issues with Simple Array Queue:

- Wasted space after multiple enqueue/dequeue (front moves forward).

Solution: Circular Queue

- Use modulo operator to wrap around.
- Track front, rear, and size (or count empty slots).

Circular Queue Visual:

text

Array: [_, _, _, _, _] (capacity = 5)

Enqueue A → [A, _, _, _, _] front=0, rear=0

Enqueue B → [A, B, _, _, _] front=0, rear=1

Dequeue → [_, B, _, _, _] front=1, rear=1

Enqueue C → [_, B, C, _, _] front=1, rear=2

... continue until wrap-around using $(\text{rear} + 1) \% \text{capacity}$

Code Snippet (Circular Queue):

Python

```
class CircularQueue:
    def __init__(self, capacity):
        self.capacity = capacity
        self.queue = [None] * capacity
        self.front = self.rear = -1
        self.size = 0

    def enqueue(self, item):
        if self.size == self.capacity:
            return "Queue Full"
        self.rear = (self.rear + 1) % self.capacity
        self.queue[self.rear] = item
        self.size += 1
        if self.front == -1:
            self.front = 0

    def dequeue(self):
        if self.size == 0:
            return "Queue Empty"
        item = self.queue[self.front]
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return item
```

Applications: Printer queues, CPU scheduling, BFS graph traversal.

3. Priority Queue (SO5)

Priority Queue: Elements dequeued based on **priority** (not just FIFO).

- Highest priority served first.
- Can be **min-heap** or **max-heap** based.

Common Implementations:

1. Unsorted Array/List (slow)
2. Sorted Array/List
3. **Binary Heap** (best for most cases — $O(\log n)$ insert/delete)
4. Fibonacci Heap (advanced)

Binary Heap (complete binary tree):

- Parent is always smaller (min-heap) or larger (max-heap) than children.
- Used in Heap Sort, Dijkstra's algorithm, etc.

Example Operations:

- Insert → bubble up
- Extract-min → replace root with last, bubble down

4. Binary Trees & Traversals (SO6)

Binary Tree: Each node has at most 2 children (left, right).

Types:

- Full, Complete, Perfect, Degenerate (linked-list like)
- Binary Search Tree (BST) — $\text{left} < \text{root} < \text{right}$

Tree Traversals (very important):

Traversal	Order	Use Case
Pre-order	Root → Left → Right	Copy tree, prefix notation
In-order	Left → Root → Right	Sorted order in BST
Post-order	Left → Right → Root	Delete tree, postfix
Level-order	Level by level (BFS)	Shortest path, serialization

Recursive Traversal Code:

Python

```
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def inorder(root):
    if root:
        inorder(root.left)
        print(root.val, end=" ")
        inorder(root.right)
```

Visual Example (Simple Binary Tree):

text

1

```
  / \
 2   3
 / \
4   5
```

In-order: 4 2 5 1 3

Pre-order: 1 2 4 5 3

Post-order: 4 5 2 3 1

5. AVL Trees / Height-Balanced Binary Trees (SO7)

Problem with normal BST: Can become skewed ($O(n)$ operations).

AVL Tree: Self-balancing BST where **balance factor** of every node is -1, 0, or 1.

Balance Factor = $\text{height}(\text{left}) - \text{height}(\text{right})$

Rotations (crucial for exams):

1. **LL Rotation** (Right rotation)

2. **RR Rotation** (Left rotation)

3. **LR Rotation** (Left-Right)

4. **RL Rotation** (Right-Left)

When to Rotate:

- After insertion/deletion if balance factor becomes +2 or -2.

Advantages: Guaranteed $O(\log n)$ height $\rightarrow$ search/insert/delete always $O(\log n)$.

Note: AVL trees are stricter than Red-Black trees (more rotations but perfectly balanced).

6. Sorting Algorithms & Algorithm Analysis (SO9–SO13) — Major Topic

Key Analysis Criteria:

- Time Complexity (Best / Average / Worst)
- Space Complexity (In-place vs Extra space)
- Stability
- Adaptive (performs better on nearly sorted data)

Comparison Table

Algorithm	Best	Average	Worst	Space	Stable?	Notes
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Simple, slow
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Good for small n
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Excellent on nearly sorted
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Stable, good for large data
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Fast in practice (pivot matters)
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Good for k-largest

Bubble Sort (already in Term 1):

- Repeatedly swap adjacent elements.

Insertion Sort:

Python

```
for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]
        j -= 1
    arr[j+1] = key
```

Merge Sort (Divide & Conquer):

- Divide → Conquer → Merge

Quick Sort (most important for analysis):

- Pivot selection critical (last, median-of-three, random).
- Partition step.

Heap Sort:

- Build max-heap → repeatedly extract max.

Counting / Radix Sort (non-comparison) — may be mentioned.

Recommended Study Approach for T2 Test

1. **Master Diagrams** — Draw trees, queues, rotations clearly.
2. **Code Tracing** — Be able to trace any traversal or sort on a small array.
3. **Complexity Proofs** — Especially for sorting algorithms.
4. **Compare & Contrast** — When to use which data structure/algorithm.
5. **Practice Exercises** from Term 1 review + new ones.

Resources:

- Any standard DSA textbook (CLRS, Goodrich, etc.)
 - Visualgo.net or GeeksforGeeks animations
 - Implement everything in Java / Python
-

Good Luck with Term 2!

Focus heavily on **Binary Trees + Sorting Analysis** — these usually carry high marks.

You can copy the above into a Word document, add more diagrams (use Draw.io or insert screenshots from Visualgo), and expand code examples as needed.

Would you like me to expand any specific section (e.g., full AVL rotation examples with diagrams, Quick Sort partition code, etc.)?